

Leading University
Department of Computer Science and
Engineering

Course Code: CSE 4114



BornoCompiler: A Native Bangla Programming
Language Compiler

Team: Symbol Table

Mumshad Chowdhury Mahdi

ID: 0182320012101083

Sharon Shahrin Mim

ID: 0182320012101064

Tasmina Rahman Chowdhury

ID: 0182320012101050

Department of Computer Science and Engineering
Leading University, Sylhet, Bangladesh

Supervisor

Alian Ahmed Ferdous

Adjunct Lecturer

Department of Computer Science and Engineering
Leading University, Sylhet, Bangladesh

September 25, 2026

BornoCompiler: A Native Bangla Programming Language Compiler



A Project
submitted to the

Department of Computer Science and Engineering
Leading University
Sylhet - 3112, Bangladesh

in partial fulfillment of the
requirements for the degree of

Bachelor of Science in Computer Science and Engineering

By

Team: Symbol Table

Mumshad Chowdhury Mahdi

ID: 0182320012101083

Sharon Shahrin Mim

ID: 0182320012101064

Tasmina Rahman Chowdhury

ID: 0182320012101050

Supervisor

Alian Ahmed Ferdous

Adjunct Lecturer

Department of Computer Science and Engineering
Leading University, Sylhet, Bangladesh

September 25, 2026

Recommendation Letter from the Project Supervisor

The project entitled “*BornoCompiler: A Native Bangla Programming Language Compiler*” submitted by the students of **Team: Symbol Table**:

1. **Mumshad Chowdhury Mahdi**, ID: 0182320012101083
2. **Sharon Shahrin Mim**, ID: 0182320012101064
3. **Tasmina Rahman Chowdhury**, ID: 0182320012101050

This is a record of research work carried out under my supervision, and I hereby approve that the report be submitted in partial fulfillment of the requirements for the award of their bachelor’s degrees.

Signature of the Supervisor:

Alian Ahmed Ferdous

Adjunct Lecturer

Department of Computer Science and Engineering

Leading University, Sylhet

Date: September 25, 2026

Certificate of Acceptance of the Project

The project entitled “*BornoCompiler: A Native Bangla Programming Language Compiler*” submitted by the students of **Team: Symbol Table**:

- 1. **Mumshad Chowdhury Mahdi**, ID: 0182320012101083
- 2. **Sharon Shahrin Mim**, ID: 0182320012101064
- 3. **Tasmina Rahman Chowdhury**, ID: 0182320012101050

on September 25, 2026

is hereby accepted in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Computer Science and Engineering.

Chairman, Defence Board

Name:

Designation:

Department of CSE

Member 1, Defence Board

Name:

Designation:

Department of CSE

Member 2, Defence Board

Name:

Designation:

Department of CSE

Abstract

This report presents the architectural specification, formal grammar, semantic validation framework, and multi-target code generation engine of **BornoCompiler**, an academic compiler designed and constructed from scratch for **Borno**—an imperative, statically typed programming language written using the native script and lexicon of the Bangla language (বাংলা). Built entirely in Java without external parser-generator dependencies, BornoCompiler implements an end-to-end six-phase pipeline: character-level Unicode lexical analysis with source-coordinate tracking, recursive-descent LL(1) parsing with panic-mode error recovery, strongly typed Abstract Syntax Tree (AST) synthesis, lexically scoped semantic analysis with static compile-time division-by-zero detection, intermediate representation via linearized Three Address Code (TAC), and dual target code generation emitting clean, indented Python 3 source code as well as low-level WebAssembly Text Format (.wat). The compiler is validated against a comprehensive test suite of 28 structured test cases and a dedicated regression test harness covering complex operator precedence, nested conditional branching, and iterative loops (যতক্ষণ, ফর). This report documents the exact implementation, class hierarchies, intermediate representation quadruples, and grammar productions as realized in the project codebase.

Contents

1	The Pitch	1
1.1	Introduction	1
1.2	Motivation	1
1.3	Problem Statement	2
1.4	Objectives	2
1.5	Real-World Use Cases	3
1.6	Target Users	3
1.7	Long-Term Vision	3
2	Language Overview	4
2.1	Introduction to Borno	4
2.2	Design Goals	4
2.3	Supported Data Types	4
2.4	Variables and Declarations	4
2.5	Operators	5
2.6	Input/Output Features	5
2.7	Conditional Statements	5
2.8	Loops	6
2.9	Other Implemented Features	6
3	Compiler Design	8
3.1	Overall Architecture	8
3.2	Compilation Pipeline	8
3.3	Lexical Analysis	8
3.4	Syntax Analysis	9
3.5	Abstract Syntax Tree	9
3.6	Semantic Analysis	9
3.7	Three Address Code Generation	9

3.8	Python Code Generation	9
3.9	WebAssembly Code Generation	9
3.10	End-to-End Pipeline	10
4	Compiler Architecture Diagram	11
5	Phase-by-Phase Implementation	12
5.1	Phase 1: Lexical Analysis	12
5.2	Phase 2: Syntax Analysis (Parser)	12
5.3	Phase 3: Semantic Analysis	13
5.4	Phase 4: Intermediate Code Generation (TAC)	13
5.5	Phase 5: Target Code Generation	13
6	UML Class Diagrams	15
6.1	Lexer and Token Module UML Class Diagram	16
6.2	Parser and AST Hierarchy UML Class Diagram	17
6.3	Semantic Analysis Module UML Class Diagram	18
6.4	Three Address Code (TAC) Module UML Class Diagram	19
6.5	Target Code Generation Module UML Class Diagram	20
6.6	Driver and Pipeline Architecture UML Class Diagram	21
7	Intermediate Representation (Three Address Code Design)	22
7.1	Principles of Three Address Code	22
7.2	Why TAC is Used in BornoCompiler	22
7.3	TACInstruction Representation	22
7.4	TACProgram Container	22
7.5	TACGenerator Lowering Logic	23
7.6	Supported TAC Operations	23
7.7	Temporary Variable Management	24
7.8	Labels and Control Flow Flattening	24
7.9	Detailed TAC Translation Example	24

8	Target Code Generation	25
8.1	Python Code Generator	25
8.2	WebAssembly (WAT) Code Generator	25
8.3	TAC-to-Python Translation Workflow	25
8.4	AST-to-WAT Translation Workflow	25
8.5	Example Target Code	26
9	Language Grammar	28
9.1	Grammar Introduction	28
9.2	Complete BNF Grammar	28
9.3	Explanation of Major Grammar Rules	30
10	Example End-to-End Compilation Walkthrough	31
10.1	Tokens Generated by Lexer	32
10.2	Abstract Syntax Tree (AST)	33
10.3	Semantic Analysis Output	33
10.4	Three Address Code Output	34
10.5	Target Code Outputs	34
11	Testing and Validation	36
11.1	Automated Test Suite (28 Test Cases)	36
11.2	Dedicated Regression Runner	38
12	Limitations	39
13	Future Work	40
14	Conclusion	41
A	Key Source Code Excerpts	43
A.1	Panic-Mode Synchronization in Parser.java	43
A.2	Static Division by Zero Detection in SemanticAnalyzer.java	43

B	Representative Borno Programs	45
B.1	Full Demo Program (valid_full_demo.brn)	45
B.2	While Loop with Nested Condition (TestRunner.java)	45

List of Figures

4.1	Overall Architecture and Data Flow of BornoCompiler	11
6.1	Lexer and Token Module UML Class Diagram	16
6.2	Parser and AST Hierarchy UML Class Diagram	17
6.3	Semantic Analysis Module UML Class Diagram	18
6.4	Three Address Code (TAC) Module UML Class Diagram	19
6.5	Target Code Generation Module UML Class Diagram	20
6.6	Compiler Pipeline and Driver UML Class Diagram	21

List of Tables

2.1	Supported Data Types in Borno	4
2.2	Operator Precedence and Associativity in Borno	5
7.1	Supported TAC Operations in BornoCompiler	23
11.1	Automated Test Suite Results (Main.java)	38

CHAPTER 1

The Pitch

1.1 Introduction

Computing and programming education are predominantly framed through the lens of Latin-derived languages. Mainstream languages—ranging from C, C++, and Java to Python and JavaScript—mandate English keywords (`let`, `int`, `if`, `while`, `print`), ASCII-based punctuation, and English standard library functions. For millions of non-English native speakers, learning computational problem-solving entails confronting two formidable intellectual hurdles simultaneously: understanding abstract algorithmic logic and navigating foreign linguistic syntax.

BornoCompiler is an authentic, university-standard compiler engineered from the ground up in Java to compile **Borno**, a programming language structured completely around the phonetic rules, vocabulary, and Unicode script of the Bangla language. Unlike superficial preprocessing scripts that simply replace keywords before piping text into an off-the-shelf runtime, BornoCompiler establishes an independent, complete compilation pipeline. It tokenizes raw Unicode text, parses syntax according to a formal context-free grammar, validates semantic consistency across lexical scopes, transforms programs into an intermediate Three Address Code (TAC) representation, and synthesizes target outputs for both high-level scripting environments (Python 3) and web/system virtual machines (WebAssembly).

1.2 Motivation

With over 300 million native speakers worldwide, Bangla is the fifth most spoken native language globally. Within Bangladesh and the state of West Bengal, India, students encountering computer science at the primary, secondary, and introductory undergraduate levels frequently experience cognitive friction when deciphering English keywords.

When a student writes:

ধরি সংখ্যা বয়স = ২০;

the conceptual alignment between natural cognition and code syntax is immediate. The student understands `ধরি` as variable initialization, `সংখ্যা` as numeric type binding, `বয়স` as the identifier, and `২০` as the literal. The cognitive overhead of linguistic translation is eliminated, allowing instructional focus to center entirely on data structures, algorithmic flow, operator precedence, and memory mutation.

1.3 Problem Statement

Prior initiatives attempting to introduce non-English educational programming typically suffer from two severe technical flaws:

1. **Text-Substitution Wrapper Flaw:** Many existing systems are naive preprocessors that perform basic regex substitutions, mapping Bangla words to English tokens before calling an external Python or C interpreter. These tools cannot provide accurate syntax error diagnostics with line/column coordinates in the native language, nor do they teach students how compilers actually analyze and lower high-level constructs.
2. **Absence of Intermediate Representation and Real Targets:** Educational toy compilers often evaluate an Abstract Syntax Tree directly via tree-walking interpreters, bypassing Intermediate Representation (IR) generation, control-flow linearization, and machine-level compilation targets.

BornoCompiler solves this challenge by implementing a complete compiler architecture from scratch, producing Three Address Code, Python 3, and WebAssembly Text Format directly from source code.

1.4 Objectives

The core technical objectives accomplished in BornoCompiler are:

- **Native Unicode Lexical Analysis:** Construct a deterministic scanner that recognizes pure Bangla Unicode code points (`\u0980–\u09FF`), handles Zero-Width Non-Joiner (ZWNJ) and Zero-Width Joiner (ZWJ) characters, tokenizes native Bangla digits (`০–৯`), and strictly enforces language purity by rejecting unauthorized Latin characters.
- **Resilient LL(1) Parsing:** Implement a recursive-descent parser equipped with panic-mode error recovery (`synchronize()`) that avoids cascading crashes and isolates syntax faults.
- **Object-Oriented AST Construction:** Produce an explicit, typed Abstract Syntax Tree capturing program semantics independently of lexical tokens.
- **Scoped Semantic Analysis:** Enforce lexical scope hierarchies with parent links, verify variable declarations and type compatibility, and evaluate expressions at compile time to catch semantic bugs like division by zero.
- **Intermediate Code Generation (TAC):** Flatten complex expressions and control flow into Three Address Code instructions using sequential temporaries (`t1, t2, ...`) and branch labels (`L1, L2, ...`).

- **Multi-Target Code Emission:** Synthesize executable Python 3 code and standard WebAssembly Text Format (.wat) capable of direct execution in web browsers and standalone WASM runtimes.

1.5 Real-World Use Cases

- **Introductory Computing in Schools:** Allowing primary and secondary schools across Bangladesh to introduce computational thinking and basic programming without requiring prior English fluency.
- **Undergraduate Compiler Pedagogy:** Serving as an open, readable, and formally structured codebase for university courses covering compiler design, parsing algorithms, scoped symbol tables, and bytecode/WASM generation.
- **Vocational Training Centers:** Providing vocational institutes in rural regions with an intuitive medium to train students in fundamental logical programming.

1.6 Target Users

- **Bangla-Speaking Beginners:** Novices seeking to grasp core programming constructs (variables, branching, loops) without language barriers.
- **Computer Science Students:** Undergraduates learning how compilers parse, analyze, optimize, and translate code.
- **Academic Instructors:** Educators who require a clean, native reference implementation for classroom instruction.

1.7 Long-Term Vision

While the current implementation fulfills all course and compiler requirements, the long-term vision for Borno envisions:

Developing a dedicated graphical IDE with real-time Bangla syntax highlighting; extending grammar support to user-defined functions, procedures, parameter passing, and arrays; and implementing direct native machine-code emission via LLVM.

CHAPTER 2

Language Overview

2.1 Introduction to Borno

Borno (বর্ণ, meaning *character* or *letter*) is a statically typed, imperative programming language. Statements terminate explicitly with a semicolon (;), code blocks are demarcated by curly braces ({ }), and expressions adhere to standard mathematical operator precedence.

2.2 Design Goals

- **Syntactic Rigor:** Borno avoids loose, dynamic typing. Every variable has a declared type, conditions must evaluate strictly to boolean values, and undeclared identifiers are rejected at compile time.
- **Linguistic Authenticity:** Keywords represent natural, mathematically accepted Bangla words (ধরি, সংখ্যা, বাক্য, যদি, নাহলে, যতক্ষণ, ফর, দেখাও).
- **Pinpoint Diagnostic Feedback:** The compiler tracks source lines and columns, reporting detailed error messages directly in Bangla.

2.3 Supported Data Types

Borno implements three concrete primitive types managed through `Type.java`:

Borno Keyword	Internal Type	Semantic Description	Example Literals
সংখ্যা	NUMBER	Double-precision float / integer	০, ১০, ২০, ৩.১৪১৬
বাক্য / লেখা	STRING	UTF-8 double-quoted string	“মাহদি”, “বাংলাদেশ”
(Inferred)	BOOLEAN	Truth value produced by comparisons	ক > ৫, বয়স >= ১৮

Table 2.1. Supported Data Types in Borno

2.4 Variables and Declarations

Variable declarations begin with the optional keyword `ধরি` (let/assume), followed by the type specifier (সংখ্যা or বাক্য), an identifier in Bangla script, an assignment operator (=), an expression, and a semicolon:

```
ধরি
সংখ্যা বয়স = ২০;ধরি
বাক্য নাম = "মাহদি";
```

Variables may also be declared without ধরি:

```
সংখ্যা
উচ্চতা = ১৭৫;
```

Reassigning previously declared variables uses the identifier name directly:

```
বয়স
= বয়স + ১;
```

2.5 Operators

Borno implements five distinct precedence tiers:

Category	Operators	Precedence	Associativity
Grouping	()	Level 5 (Highest)	Left-to-Right
Unary	-, +, !, না	Level 4	Right-to-Left
Multiplicative	*, /, %	Level 3	Left-to-Right
Additive	+, -	Level 2	Left-to-Right
Relational	==, !=, <, >, <=, >=	Level 1 (Lowest)	Left-to-Right
Assignment	=	Statement Level	Right-to-Left

Table 2.2. Operator Precedence and Associativity in Borno

2.6 Input/Output Features

Terminal output is implemented through the দেখাও (show/display) statement:

```
দেখাওবয়স
();দেখাওস্বাগতম
(" বর্ণ কম্পাইলারে");দেখাওক
( * ২ + ৫);
```

2.7 Conditional Statements

Conditional branching is provided via যদি (if) and optional নাহলে (else) blocks:

```

যদি
বয়স( >= ১৮) {
    ধরি বাক্য অবস্থা = প্রাপ্তবয়স্ক"";
    দেখাওঅবস্থা();
} নাহলে {
    ধরি বাক্য অবস্থা = অপ্রাপ্তবয়স্ক"";
    দেখাওঅবস্থা();
}

```

2.8 Loops

Borno supports both conditional while-loops and iterative for-loops:

- **While Loop (যতক্ষণ):**

```

ধরি
সংখ্যা ক = ০; যতক্ষণ
ক( < ৫) {
    দেখাওক();
    ক = ক + ১;
}

```

- **For Loop (ফর / for):**

```

ফর
ধরি( সংখ্যা ক = ০; ক < ৫; ক = ক + ১) {
    দেখাওক();
}

```

2.9 Other Implemented Features

- **Single-Line Comments:** Using #:

```

# এটি একটি একক লাইনের মন্তব্যধরি
সংখ্যা মান = ৫০; # ইনলাইন মন্তব্য

```

- **String Concatenation:** The + operator performs concatenation if either operand is of type STRING:

```

ধরি
বাক্য বার্তা = হ্যালো"" + বাংলাদেশ"";

```

- **Static Division by Zero Detection:** Compile-time constant evaluation flags expressions like $\text{!0} / 0$ or divisions by known zero-valued variables before code emission.

CHAPTER 3

Compiler Design

3.1 Overall Architecture

BornoCompiler follows a classic multi-phase compiler pipeline:

Source Code → **Lexer** → **Parser** → **AST** → **Semantic Analyzer** → **TAC Generator** →
Python / Wasm Generators → **Target Output**

3.2 Compilation Pipeline

Compilation proceeds through strict phases. If an earlier phase encounters errors, subsequent phases are skipped:

1. **Ingestion:** The source text is loaded as a UTF-8 string.
2. **Lexical Analysis:** Characters are converted into tokens; unrecognized or unauthorized characters (e.g., English digits) trigger lexical errors.
3. **Syntax Analysis:** Tokens are parsed into an Abstract Syntax Tree. Syntax errors trigger synchronization recovery.
4. **Semantic Analysis:** The AST is checked against a scoped symbol table. Type mismatches, duplicate declarations, and undeclared variables are flagged.
5. **TAC Generation:** The validated AST is lowered into linear Three Address Code.
6. **Target Code Generation:**
 - **Python Backend:** Translates TAC instructions into indented Python 3.
 - **WebAssembly Backend:** Translates the AST into WebAssembly Text Format (.wat).

3.3 Lexical Analysis

Implemented in `Lexer.java`, the scanner processes input character-by-character. It tracks position, `currentChar`, `line`, and `col`. It utilizes `NumberHelper.java` to validate Bangla digits (০–৯). Any ASCII digits (0–9) or English keywords/identifiers are rejected with clear diagnostic error messages.

3.4 Syntax Analysis

Implemented in `Parser.java`, the parser uses an LL(1) recursive-descent approach with precedence climbing. It includes panic-mode error recovery via `synchronize()`, skipping tokens until statement boundaries (like `;`, `}`, or declaration keywords) are reached.

3.5 Abstract Syntax Tree

All nodes derive from `ASTNode.java`. Statements include `AssignmentNode`, `PrintNode`, `IfNode`, `WhileNode`, `ForNode`, `BlockNode`, and `ProgramNode`. Expressions include `BinaryExpressionNode`, `UnaryExpressionNode`, `LiteralNode`, and `VariableNode`.

3.6 Semantic Analysis

Implemented in `SemanticAnalyzer.java`, this phase verifies type compatibility, maintains hierarchical lexical scopes via `SymbolTable.java`, verifies boolean conditions for loops and branches, and evaluates constant expressions to catch compile-time division by zero.

3.7 Three Address Code Generation

Implemented in `TACGenerator.java`, this phase lowers tree structures into linear sequences of `TACInstruction` objects stored in a `TACProgram`. Subexpressions are assigned to sequential temporaries (`t1`, `t2`, ...) and jumps target generated labels (`L1`, `L2`, ...).

3.8 Python Code Generation

Implemented in `PythonGenerator.java`, the Python backend converts linear TAC instructions into structured Python 3. It pattern-matches conditional jumps to generate indented `if/else` and `while` blocks, converts Bangla numbers to ASCII digits, and configures UTF-8 standard output.

3.9 WebAssembly Code Generation

Implemented in `WasmGenerator.java`, this backend translates the AST into WebAssembly Text Format (`.wat`). Local variables are mapped to `i32` registers, strings are allocated in linear memory data segments, and expressions are emitted as stack operations (`i32.add`, `i32.sub`, `call $print_i32`, `call $print_str`).

3.10 End-to-End Pipeline

The entry point in `Main.java` provides an interactive CLI menu that supports single-phase debugging, automated test suite execution, and direct execution of generated Python and WebAssembly files.

CHAPTER 4

Compiler Architecture Diagram

Figure 4.1 illustrates the complete compilation flow from source text to target execution:

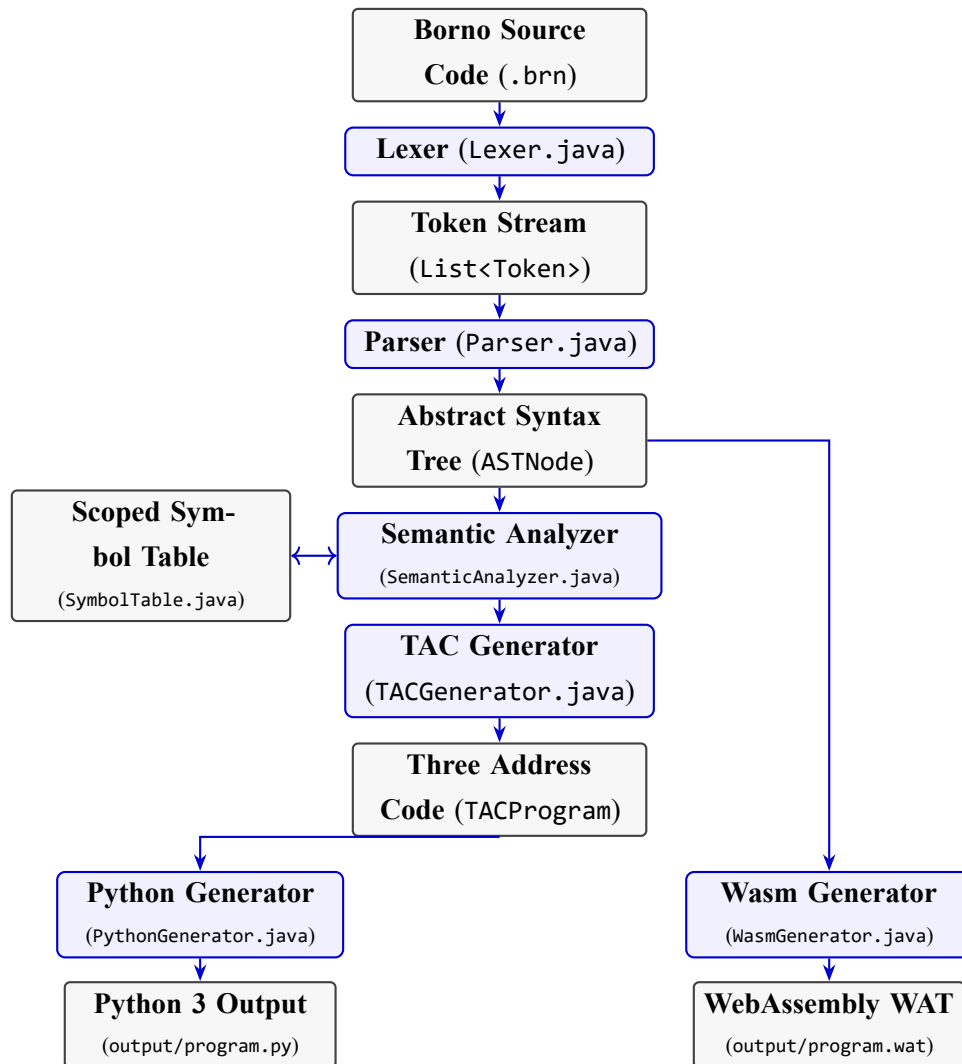


Figure 4.1. Overall Architecture and Data Flow of BornoCompiler

CHAPTER 5

Phase-by-Phase Implementation

5.1 Phase 1: Lexical Analysis

- **Purpose:** Converts raw Bangla source text into a stream of tokens while filtering whitespace and comments.
- **Input:** Source code string.
- **Processing:** Character-by-character scanning with position tracking (`line`, `col`). It enforces native Bangla Unicode validation and maps keyword strings to `TokenType` enums.
- **Output:** A `List<Token>` terminated by `TokenType.EOF`.
- **Important Classes:** `Lexer`, `Token`, `TokenType`, `NumberHelper`.
- **Important Methods:** `nextToken()`, `tokenize()`, `readNumber()`, `readString()`, and `readIdentifierOrKeyword()`.

5.2 Phase 2: Syntax Analysis (Parser)

- **Purpose:** Validates token sequences against the Borno grammar and constructs an Abstract Syntax Tree.
- **Input:** `List<Token>` produced by the `Lexer`.
- **Processing:** LL(1) recursive-descent parsing with precedence climbing. Uses panic-mode error recovery (`synchronize()`) to avoid cascading failures.
- **Output:** A `ProgramNode` representing the program root.
- **Important Classes:** `Parser`, `ASTNode`, `ASTPrinter`.
- **Important Methods:** `parseProgram()`, `parseStatement()`, `parseDeclare()`, `parseIf()`, `parseWhile()`, `parseFor()`, `consume()`, and `synchronize()`.

5.3 Phase 3: Semantic Analysis

- **Purpose:** Validates language semantics, ensures variables are declared before use, and checks type compatibility.
- **Input:** ProgramNode AST.
- **Processing:** Traverses the AST while managing scoped symbol tables. Evaluates constant expressions at compile time to detect division by zero.
- **Output:** Semantic validation logs, populated symbol table, or semantic error exceptions.
- **Important Classes:** SemanticAnalyzer, SymbolTable, Type.
- **Important Methods:** analyze(), analyzeAssignment(), analyzeIf(), analyzeWhile(), analyzeFor(), and evaluateConstant().

5.4 Phase 4: Intermediate Code Generation (TAC)

- **Purpose:** Flattens the hierarchical AST into a linear stream of Three Address Code instructions.
- **Input:** Validated ProgramNode AST.
- **Processing:** Generates sequential temporary variables (t1, t2, ...) and control-flow labels (L1, L2, ...).
- **Output:** A TACProgram containing TACInstruction objects.
- **Important Classes:** TACGenerator, TACProgram, TACInstruction.
- **Important Methods:** generate(), generateStatement(), newTemp(), newLabel(), and writeToFile().

5.5 Phase 5: Target Code Generation

- **Purpose:** Emits runnable Python 3 and WebAssembly Text Format source files.
- **Input:** TACProgram (for Python) and ProgramNode AST (for WebAssembly).
- **Processing:**
 - *Python:* Pattern-matches TAC jumps to reconstruct indented if and while structures; translates operators and converts Bangla digits.

- *WebAssembly*: Allocates local variables, writes strings into linear memory data segments, and emits stack bytecode instructions.
- **Output**: `output/program.py`, `output/program.wat`, and compiled `output/program.wasm`.
- **Important Classes**: `PythonGenerator`, `WasmGenerator`.
- **Important Methods**: `PythonGenerator.generate()`, `WasmGenerator.generate()`, and `compileWatToWasm()`.

CHAPTER 6

UML Class Diagrams

This chapter details the object-oriented structure of BornoCompiler. The system architecture is organized into modular subsystems covering lexical scanning, syntax tree hierarchies, scoped semantic environments, three-address intermediate code representations, multi-target code synthesis, and the interactive compiler driver.

The design adheres to classical compiler engineering principles, isolating phase responsibilities while establishing clear data contracts between pipeline stages. Each subsystem is documented with its primary classes, fields, visibility qualifiers, and method contracts.

The subsequent sections dedicate a comprehensive page to each compiler subsystem, providing detailed structural descriptions alongside reserved spaces for visual UML class diagrams:

- **Section 6.1:** *Lexer and Token Module* (Figure 6.1) — Token classification, coordinate tracking, and Bangla numeric conversion.
- **Section 6.2:** *Parser and AST Hierarchy* (Figure 6.2) — Composite AST nodes, statements, expressions, and recursive descent.
- **Section 6.3:** *Semantic Analysis Module* (Figure 6.3) — Scoped symbol resolution, static types, and compile-time evaluation.
- **Section 6.4:** *Three Address Code Module* (Figure 6.4) — Linear intermediate representation, temporary registers, and jump labels.
- **Section 6.5:** *Target Code Generation Module* (Figure 6.5) — Python 3 structured generation and WebAssembly Text Format emission.
- **Section 6.6:** *Driver and Pipeline Architecture* (Figure 6.6) — Central CLI runner, runtime execution harness, and test runner.

Interactive visual diagrams and their source representations can be accessed in `diagrams/view_uml.html`.

6.1 Lexer and Token Module UML Class Diagram

The lexical analysis subsystem converts the raw Bangla source string into a stream of strongly typed tokens with line and column coordinates. It comprises four primary components: **TokenType** (enumerates 24 token kinds including language keywords, literals, arithmetic/relational operators, and delimiters), **Token** (encapsulates classification type, string value, and 1-based source coordinates), **NumberHelper** (static utility routines for bidirectional conversion between Bangla and ASCII digits and numeric validation), and **Lexer** (the scanning engine managing coordinate pointers, Unicode character classification, and lexical error reporting).

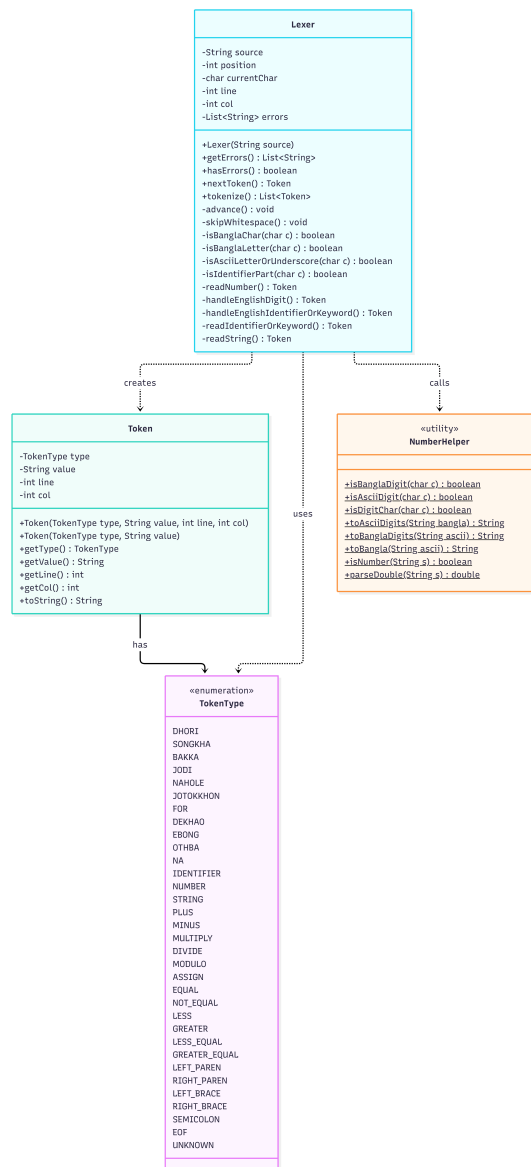


Figure 6.1. Lexer and Token Module UML Class Diagram

6.2 Parser and AST Hierarchy UML Class Diagram

The syntactic analysis subsystem parses the token stream into a typed Abstract Syntax Tree (AST) using an LL(1) recursive-descent strategy. The AST hierarchy is rooted at the abstract **ASTNode** base class, extended by statement nodes (ProgramNode, BlockNode, AssignmentNode, IfNode, WhileNode, ForNode, PrintNode) and expression nodes (BinaryExpressionNode, UnaryExpressionNode, VariableNode, LiteralNode). The **Parser** coordinates grammar productions, operator precedence, and panic-mode synchronization, while **ASTPrinter** provides hierarchical tree visualization.

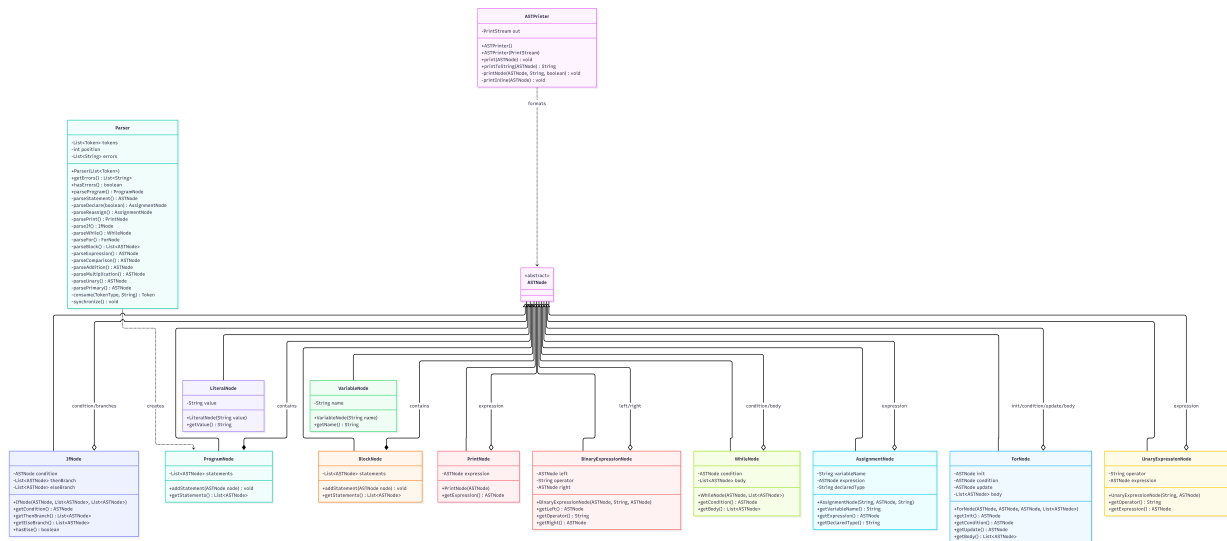


Figure 6.2. Parser and AST Hierarchy UML Class Diagram

6.3 Semantic Analysis Module UML Class Diagram

The semantic analysis subsystem verifies static type safety, identifier declaration scopes, and constant expressions. Key components include **Type** (enumerates static types NUMBER, STRING, BOOLEAN, and UNKNOWN with native Bangla diagnostic strings), **SymbolTable** (hierarchical scoped environment storing symbol types, initialization states, constant values, and parent scope pointers), and **SemanticAnalyzer** (walks the AST to perform scoped validation, type checking, and compile-time division-by-zero detection).

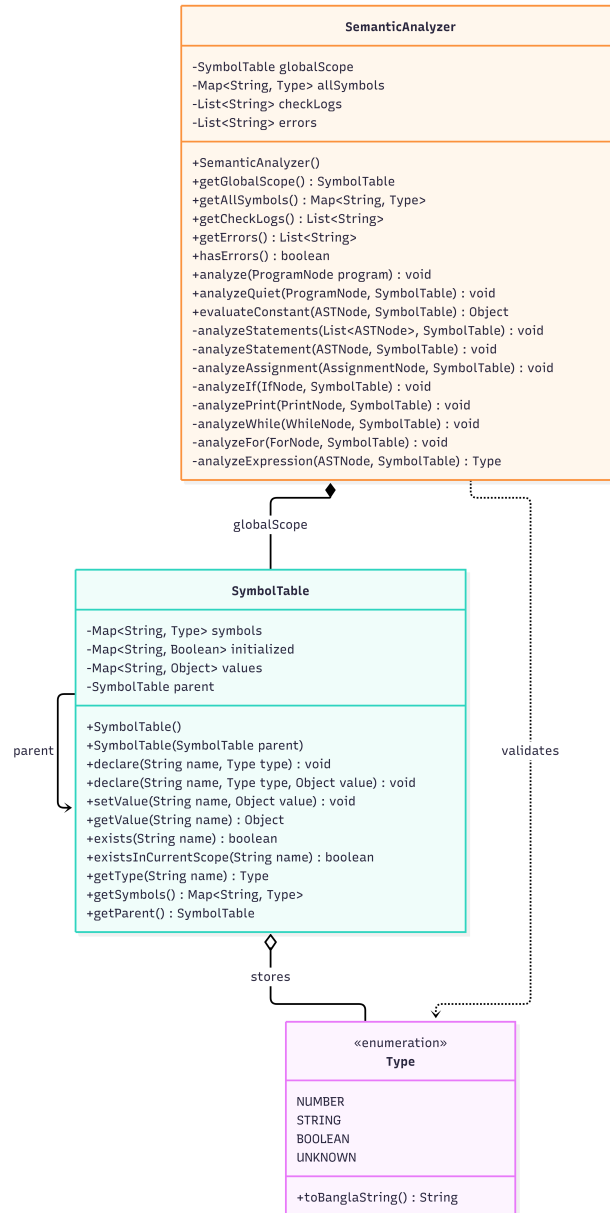


Figure 6.3. Semantic Analysis Module UML Class Diagram

6.4 Three Address Code (TAC) Module UML Class Diagram

The intermediate representation subsystem linearizes nested expressions and hierarchical control flow into three-address quadruples. The module comprises **OpType** (defines the eight instruction variants including binary, unary, assignment, branching, and labels), **TACInstruction** (encapsulates operator, operands, result, and static factory constructors), **TACProgram** (container managing the sequential instruction sequence and formatting printable output), and **TACGenerator** (lowers AST constructs into temporaries `t1`, `t2` and branch labels `L1`, `L2`).

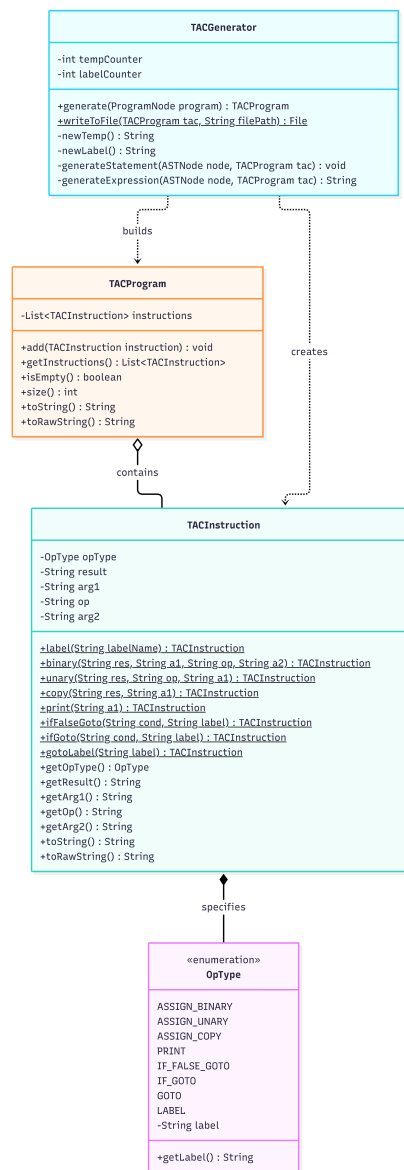


Figure 6.4. Three Address Code (TAC) Module UML Class Diagram

6.5 Target Code Generation Module UML Class Diagram

The code generation subsystem translates intermediate representations into runnable target formats. It comprises **PythonGenerator** (converts TAC instruction sequences into clean, indented Python 3 source code by reconstructing structured `if`, `else`, and `while` blocks and converting Bangla numerals to ASCII digits) and **WasmGenerator** (traverses the AST to emit stack-oriented WebAssembly Text Format `.wat`, manages local variable indices, allocates linear memory for string constants, and invokes `wat2wasm` to generate binary `.wasm` packages).

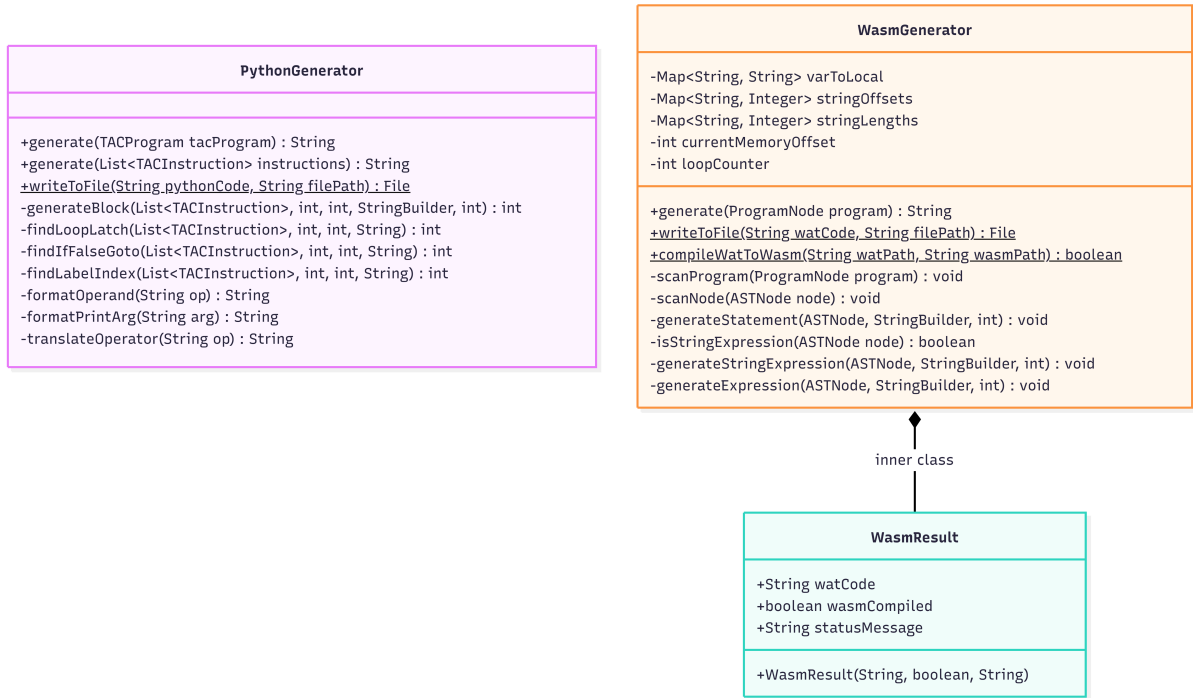


Figure 6.5. Target Code Generation Module UML Class Diagram

6.6 Driver and Pipeline Architecture UML Class Diagram

The compiler driver and test pipeline coordinate all compilation stages and provide automated regression testing. The module features **Main** (the primary interactive command-line interface managing stage-by-stage execution from lexing to Python and WebAssembly execution, custom source input handling, and test-suite invocation) and **TestRunner** (the standalone automated test harness that executes the 28 comprehensive test cases and validates compiler output across all subsystems).

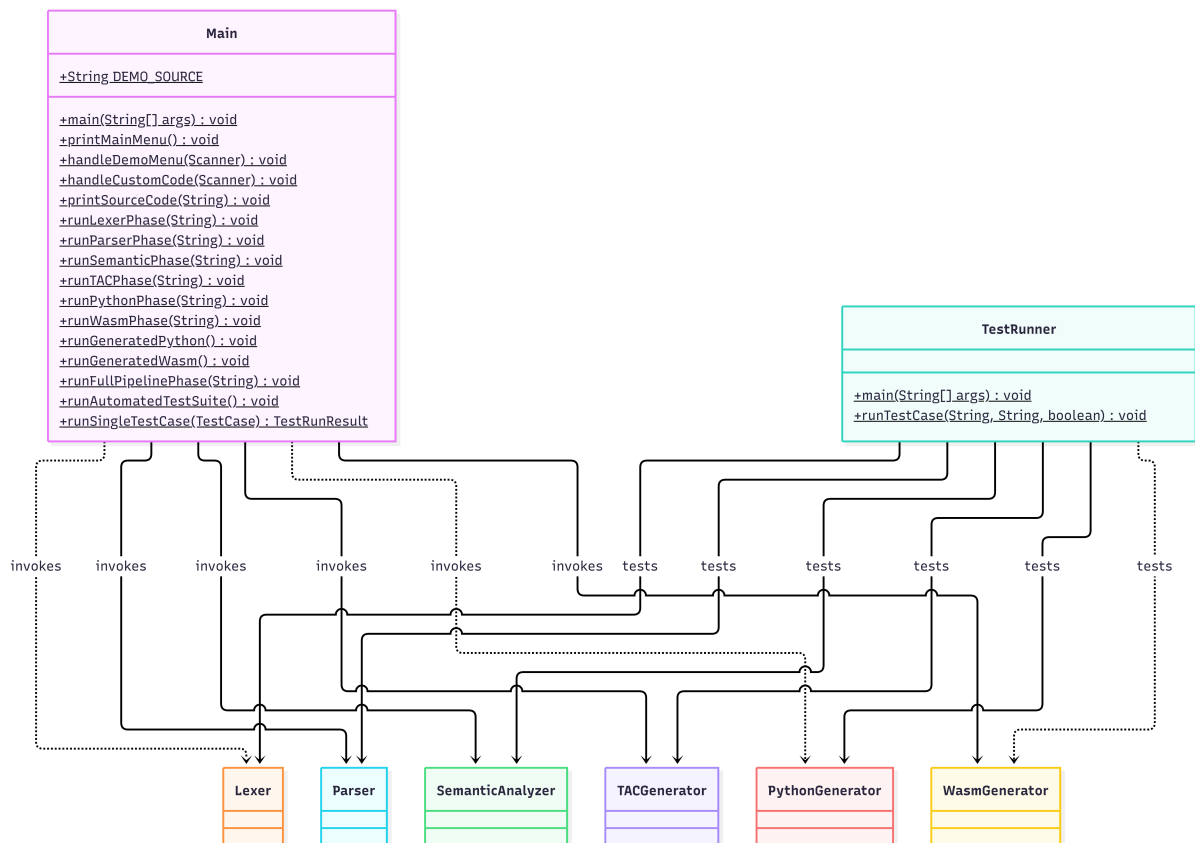


Figure 6.6. Compiler Pipeline and Driver UML Class Diagram

CHAPTER 7

Intermediate Representation (Three Address Code Design)

7.1 Principles of Three Address Code

Three Address Code (TAC) is an intermediate representation where instructions have at most one operator and at most three addresses:

$$\text{Result} = \text{Operand}_1 \quad \text{Operator} \quad \text{Operand}_2$$

7.2 Why TAC is Used in BornoCompiler

1. **Decoupling:** Separates language parsing from target code emission.
2. **Control-Flow Normalization:** High-level loops and branches are simplified into conditional and unconditional jumps.
3. **Target Extensibility:** Adding new target architectures requires only reading the linear TAC instruction stream.

7.3 TACInstruction Representation

Implemented in `TACInstruction.java`, each instruction stores:

- `opType`: The instruction category from `OpType`.
- `result`: Target variable, allocated temporary, or jump label.
- `arg1`: First source operand.
- `op`: Operator string.
- `arg2`: Second source operand (if applicable).

7.4 TACProgram Container

`TACProgram.java` stores instructions and provides two serialization outputs:

1. Annotated Bangla Output: Used in compiler educational logs:

```
১
. [Copy] বয়স = ২০২
. [BinOp] t1 = বয়স >= ১৮৩
. [IfFalseGoto] শর্ত t1 মিথ্যা হলে L1-এ যাও
```

2. Raw TAC Output: Formatted as conventional compiler IR:

```
বয়স = ২০
t1 = বয়স >= ১৮
ifFalse t1 goto L1
```

7.5 TACGenerator Lowering Logic

TACGenerator.java lowers AST nodes as follows:

- **Assignment:** Evaluates the expression and emits ASSIGN_COPY.
- **Print:** Evaluates the expression and emits PRINT.
- **If-Else:** Evaluates the condition, emits ifFalseGoto targeting the else-label, lowers the then-block, emits goto targeting the end-label, and places labels appropriately.
- **While Loop:** Emits a start label, evaluates the condition, emits an ifFalseGoto to the exit label, lowers body statements, emits an unconditional jump back to the start label, and places the exit label.

7.6 Supported TAC Operations

OpType Enum	Raw Syntax	Bangla Annotated Format
ASSIGN_BINARY	t1 = a + b	[BinOp] t1 = a + b
ASSIGN_UNARY	t1 = -a	[Unary] t1 = -a
ASSIGN_COPY	x = t1	[Copy] x = t1
PRINT	print x	[Print] দেখাও(x)
IF_FALSE_GOTO	ifFalse t1 goto L1	[IfFalseGoto] শর্ত t1 মিথ্যা হলে L1-এ যাও
IF_GOTO	if t1 goto L1	[IfGoto] শর্ত t1 সত্য হলে L1-এ যাও
GOTO	goto L2	[Goto] যাও L2
LABEL	L1:	[Label] L1:

Table 7.1. Supported TAC Operations in BornoCompiler

7.7 Temporary Variable Management

Temporaries are allocated sequentially via `newTemp()` as `t1`, `t2`, `t3`, ...to hold subexpression results during arithmetic evaluation.

7.8 Labels and Control Flow Flattening

Symbolic labels are generated via `newLabel()` as `L1`, `L2`, `L3`, ...to mark branch targets and loop boundaries.

7.9 Detailed TAC Translation Example

For the input:

```
ধরি  
সংখ্যা ক = ১০ + ২০ * ৫;
```

The multiplication subexpression is evaluated first due to precedence climbing, yielding:

```
১  
. [BinOp] t1 = ২০ * ৫২  
. [BinOp] t2 = ১০ + t1৩  
. [Copy] ক = t2
```

CHAPTER 8

Target Code Generation

8.1 Python Code Generator

`PythonGenerator.java` translates TAC instructions into executable Python 3. It detects branch and jump patterns to generate properly indented blocks, converts Bangla numbers using `NumberHelper.toAsciiDigits()`, and configures UTF-8 encoding on standard output.

8.2 WebAssembly (WAT) Code Generator

`WasmGenerator.java` translates the AST into WebAssembly Text Format (`.wat`). Variables become local i32 registers, strings are placed in linear memory data segments, and expressions are emitted as stack operations.

8.3 TAC-to-Python Translation Workflow

1. Pattern-matches TAC jump instructions to identify block structures.
2. Recursively generates indented blocks for `if`, `else`, and `while` constructs.
3. Translates operators (এবং $\rightarrow$ `and`, অথবা $\rightarrow$ `or`, না $\rightarrow$ `not`).
4. Writes formatted code to `output/program.py`.

8.4 AST-to-WAT Translation Workflow

1. Scans the AST to identify all variables and string literals.
2. Emits module headers, runtime imports (`$print_i32`, `$print_str`), and memory exports.
3. Emits data segments for string constants with null terminators.
4. Emits function `$main` containing local variable declarations and statement instructions.
5. Invokes `wat2wasm` if present to compile `output/program.wasm`.

8.5 Example Target Code

For the program:

```
ধরি
সংখ্যা বয়স = ২০; যদি
বয়স( >= ১৮) {
    ধরি সংখ্যা অবস্থা = ১;
} নাহলে {
    ধরি সংখ্যা অবস্থা = ০;
}
```

Generated Python 3 (output/program.py):

```
# -*- coding: utf-8 -*-
import sys
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8') বয়স

= 20
t1 = বয়স >= 18
if t1:
    অবস্থা = 1
else:
    অবস্থা = 0
```

Generated WebAssembly Text Format (output/program.wat):

```
(module
  (import "env" "print_i32" (func $print_i32 (param i32)))
  (import "env" "print_str" (func $print_str (param i32 i32)))
  (memory (export "memory") 1)

  (func $main (export "main")
    (local $var_0 i32) ;; বয়স
    (local $var_1 i32) ;; অবস্থা

    i32.const 20
    local.set $var_0
    local.get $var_0
    i32.const 18
    i32.ge_s
    if
```

```
    i32.const 1
    local.set $var_1
  else
    i32.const 0
    local.set $var_1
  end
)
)
```

CHAPTER 9

Language Grammar

9.1 Grammar Introduction

The formal grammar of Borno is derived directly from `Parser.java`. It is an unambiguous, context-free grammar designed for deterministic LL(1) parsing.

9.2 Complete BNF Grammar

```
<program> ::= <statement_list> EOF

<statement_list> ::= <statement> <statement_list>
                    | epsilon

<statement> ::= <declare_stmt>
                | <reassign_stmt>
                | <print_stmt>
                | <if_stmt>
                | <while_stmt>
                | <for_stmt>

<declare_stmt> ::= ধরি'' <type_specifier> IDENTIFIER '=' <expression> ';'
                | <type_specifier> IDENTIFIER '=' <expression> ';'

<type_specifier> ::= সংখ্যা''
                  | বাক্য''
                  | লেখা''

<reassign_stmt> ::= IDENTIFIER '=' <expression> ';'

<print_stmt> ::= দেখাও'' '(' <expression> ')' ';'

<if_stmt> ::= যদি'' '(' <expression> ')' <block> <else_opt>

<else_opt> ::= নাহলে'' <block>
             | epsilon

<while_stmt> ::= যতক্ষণ'' '(' <expression> ')' <block>

<for_stmt> ::= <for_keyword> '(' <for_init> <for_cond> <for_update> ')' <block>
```

```

<for_keyword> ::= ফর'' | 'for' | জন্য'' | প্রতি'' | প্রতিটি''

<for_init> ::= <declare_stmt>
            | <reassign_stmt>
            | <expression> ';'
            | ';'

<for_cond> ::= <expression> ';'
            | ';'

<for_update> ::= IDENTIFIER '=' <expression>
              | <expression>
              | epsilon

<block> ::= '{' <statement_list> '}'

<expression> ::= <comparison>

<comparison> ::= <addition> ( <rel_op> <addition> )*

<rel_op> ::= '==' | '!=' | '<' | '>' | '<=' | '>='

<addition> ::= <multiplication> ( <add_op> <multiplication> )*

<add_op> ::= '+' | '-'

<multiplication> ::= <unary> ( <mul_op> <unary> )*

<mul_op> ::= '*' | '/' | '%'

<unary> ::= '-' <unary>
          | '+' <unary>
          | '!' <unary>
          | না'' <unary>
          | <primary>

<primary> ::= NUMBER
          | STRING
          | IDENTIFIER
          | '(' <expression> ')'

```

9.3 Explanation of Major Grammar Rules

- **Precedence Hierarchy:** Stratified productions enforce standard algebraic order of operations (comparison $\rightarrow$ addition $\rightarrow$ multiplication $\rightarrow$ unary $\rightarrow$ primary).
- **Left Associativity:** Binary operators are parsed iteratively in `Parser.java` using loops, establishing left-associative tree construction.
- **Right Associativity:** Unary operators are parsed via direct recursion, establishing right-associative binding.

CHAPTER 10

Example End-to-End Compilation Walkthrough

This chapter traces the complete end-to-end compilation lifecycle of a representative Borno source program. The walkthrough demonstrates how each phase of the BornoCompiler pipeline transforms the program—progressing sequentially from raw source text to lexical tokens, an Abstract Syntax Tree (AST), scoped symbol resolution, linear Three Address Code (TAC), and final target code generation in Python 3 and WebAssembly.

Sample Source Program

The example program declares two integer variables (ক and খ), computes an arithmetic expression with nested operator precedence (৫ * ২ before addition with ক), and prints the resulting value using the native দেখাও statement:

```
ধরি সংখ্যা ক = ১০;  
ধরি সংখ্যা খ = ক + ৫ * ২;  
দেখাও(খ);
```

The following sections illustrate the concrete intermediate representations and outputs synthesized at each compiler milestone.

10.1 Tokens Generated by Lexer

DHORI	:	'ধরি'	(লাইন ১)
SONGKHA	:	'সংখ্যা'	(লাইন ১)
IDENTIFIER	:	'ক'	(লাইন ১)
ASSIGN	:	'='	(লাইন ১)
NUMBER	:	'১০'	(লাইন ১)
SEMICOLON	:	';'	(লাইন ১)
DHORI	:	'ধরি'	(লাইন ২)
SONGKHA	:	'সংখ্যা'	(লাইন ২)
IDENTIFIER	:	'খ'	(লাইন ২)
ASSIGN	:	'='	(লাইন ২)
IDENTIFIER	:	'ক'	(লাইন ২)
PLUS	:	'+'	(লাইন ২)
NUMBER	:	'৫'	(লাইন ২)
MULTIPLY	:	'*'	(লাইন ২)
NUMBER	:	'২'	(লাইন ২)
SEMICOLON	:	';'	(লাইন ২)
DEKHAO	:	'দেখাও'	(লাইন ৩)
LEFT_PAREN	:	'('	(লাইন ৩)
IDENTIFIER	:	'খ'	(লাইন ৩)
RIGHT_PAREN	:	')'	(লাইন ৩)
SEMICOLON	:	';'	(লাইন ৩)
EOF	:	'	(লাইন ৪)

10.2 Abstract Syntax Tree (AST)

```
PROGRAM
|
+- DECLARATION
|   +- TYPE : সংখ্যা
|   +- NAME : ক
|   '- VALUE : ১০
|
+- DECLARATION
|   +- TYPE : সংখ্যা
|   +- NAME : খ
|   '- VALUE : (ক + (৫ * ২))
|
'- PRINT
'- EXPR : খ
```

10.3 Semantic Analysis Output

Symbol Table:

Name	Type	Initialized
ক	সংখ্যা	হ্যাঁ
খ	সংখ্যা	হ্যাঁ

Semantic Check Logs:

```
[OK] ক -> সংখ্যা
[OK] খ -> সংখ্যা
[OK] PRINT statement valid
```

10.4 Three Address Code Output

```
১. [Copy] ƙ = ১০
২. [BinOp] t1 = ৫ * ২
৩. [BinOp] t2 = ƙ + t1
৪. [Copy] ৎ = t2
৫. [Print] দেখাও(ৎ)
```

10.5 Target Code Outputs

Python 3 Output (output/program.py):

```
# -*- coding: utf-8 -*-
import sys
if hasattr(sys.stdout, 'reconfigure'):
    sys.stdout.reconfigure(encoding='utf-8')
ƙ = 10
t1 = 5 * 2
t2 = ƙ + t1
ৎ = t2
print(ৎ)
```

WebAssembly Output (output/program.wat):

```
(module
  (import "env" "print_i32" (func $print_i32 (param i32)))
  (import "env" "print_str" (func $print_str (param i32 i32)))
  (memory (export "memory") 1)

  (func $main (export "main")
    (local $var_0 i32) ;; ƙ
    (local $var_1 i32) ;; ৎ
    i32.const 10
    local.set $var_0
    local.get $var_0
    i32.const 5
    i32.const 2
    i32.mul
    i32.add
    local.set $var_1
    local.get $var_1
    call $print_i32
  )
)
```


CHAPTER 11

Testing and Validation

11.1 Automated Test Suite (28 Test Cases)

The compiler includes an automated test harness embedded directly in `Main.java`. Table 11.1 summarizes the test cases and their execution outcomes:

ID	Test Name	Input Snippet	Expected Outcome	Result
1	Bangla Digits Accepted	ধরি সংখ্যা বয়স = ২০;	Success	PASS
2	English Digits Rejected	ধরি সংখ্যা বয়স = 20;	Lexical Error	PASS
3	Bangla Identifier Accepted	ধরি সংখ্যা বয়স = ২০;	Success	PASS
4	English Identifier Rejected	ধরি সংখ্যা age = ২০;	Lexical Error	PASS
5	Mixed Identifier Rejected	ধরি সংখ্যা বয়স123 = ২০;	Lexical Error	PASS
6	English Keyword Rejected	if (বয়স >= ১৮) { ... }	Lexical Error	PASS
7	Mixed Keyword Rejected	dhori সংখ্যা বয়স = ২০;	Lexical Error	PASS
8	Valid Multi-statement Demo	Complete demo program	Success	PASS
9	Type Mismatch	ধরি সংখ্যা বয়স = "মাহদি";	Semantic Error	PASS
10	Division by Zero	ধরি সংখ্যা ফলাফল = ১০ / ০;	Semantic Error	PASS

ID	Test Name	Input Snippet	Expected Outcome	Result
11	Undeclared Variable in Print	দেখাও(নাম);	Semantic Error	PASS
12	Undeclared Variable in Expr	ধরি সংখ্যা ফলাফল = বয়স + ১০;	Semantic Error	PASS
13	Duplicate Declaration	ধরি সংখ্যা বয়স = ২০; ...	Semantic Error	PASS
14	Non-Boolean IF Condition	যদি (বয়স) { ... }	Semantic Error	PASS
15	Missing Semicolon	ধরি সংখ্যা বয়স = ২০	Syntax Error	PASS
16	Invalid Character	ধরি সংখ্যা \$বয়স = ২০;	Lexical Error	PASS
17	Unclosed String	ধরি বাক্য বার্তা = "অসমাপ্ত;	Lexical Error	PASS
18	Valid Number & String	Variable declarations	Success	PASS
19	Operator Precedence	১০ + ২০ * ৩ - ৫	Success	PASS
20	String Print Statement	String output via দেখাও	Success	PASS
21	Nested IF and Scoping	Block-scoped variables	Success	PASS
22	Full Demo Program	Conditional branching demo	Success	PASS
23	Valid Unary Expression	ধরি সংখ্যা ক = -৫;	Success	PASS
24	Valid While Loop	যতক্ষণ (ক < ৫) { ... }	Success	PASS
25	Valid For Loop	for (ক=০; ক<৫; ক=ক+১)	Success	PASS
26	Nested Loop in Conditional	While inside if condition	Success	PASS

ID	Test Name	Input Snippet	Expected Outcome	Result
27	Non-Boolean While Condition	যতক্ষণ (ক) { ... }	Semantic Error	PASS
28	Intentional Failure Test	Valid division evaluated against expected error	Harness verification	EXP. FAIL

Table 11.1. Automated Test Suite Results (Main.java)

11.2 Dedicated Regression Runner

`TestRunner.java` provides regression verification across 12 targeted scenarios focusing on loop latches, unary operators, and nested scope isolation.

CHAPTER 12

Limitations

To maintain technical accuracy, the current limitations of the implementation are documented below:

1. **No User-Defined Functions:** Programs consist of a single top-level sequence of statements; functions and procedures are not yet supported.
2. **No Array or Composite Structures:** Variables are limited to scalar types (NUMBER, STRING, BOOLEAN).
3. **WebAssembly Integer Truncation:** In `WasmGenerator.java`, numeric values are cast to WebAssembly i32 registers, truncating fractional values in WASM output (Python retains full double-precision support).
4. **No Interactive Input Construct:** Terminal input statements (such as `ইনপুট()`) are not yet implemented.

CHAPTER 13

Future Work

Planned future enhancements include:

- **Function Declarations:** Supporting user-defined functions with parameters and return values.
- **Arrays and Lists:** Adding support for indexed arrays (তালিকা).
- **Interactive Input:** Implementing an input construct for reading values from standard input.
- **Direct Bytecode Emission:** Compiling directly to JVM bytecode (.class) or native binaries via LLVM.
- **Bangla IDE:** Creating a graphical editor with syntax highlighting and inline diagnostic messages.

CHAPTER 14

Conclusion

BornoCompiler demonstrates a complete, working implementation of compiler construction principles applied to a native Bangla programming language. Built entirely from scratch in Java, the project implements an end-to-end six-phase pipeline: lexical scanning with coordinate tracking, recursive-descent LL(1) parsing with error recovery, Abstract Syntax Tree construction, scoped semantic analysis with compile-time constant evaluation, intermediate Three Address Code generation, and multi-target code emission for both Python 3 and WebAssembly.

Validated through automated test suites and sample programs, the compiler demonstrates that programming languages can be designed and implemented cleanly in non-Latin scripts, helping bridge educational gaps for Bangla-speaking computer science students.

Bibliography

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011.
- [3] R. Nystrom, *Crafting Interpreters*. Genever Benning, 2021.
- [4] The Unicode Consortium, *The Unicode Standard, Version 15.0: Bengali Script Block (Range: 0980–09FF)*. Unicode, Inc., 2023.
- [5] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in *Proc. 38th ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.
- [6] Python Software Foundation, *Python 3 Language Reference: Execution Model and Grammar*. Available: <https://docs.python.org/3/reference/>, 2023.

APPENDIX A

Key Source Code Excerpts

A.1 Panic-Mode Synchronization in Parser.java

```
private void synchronize() {
    while (!check(TokenType.EOF)) {
        if (check(TokenType.SEMICOLON)) {
            advance();
            return;
        }
        if (check(TokenType.RIGHT_BRACE)) {
            return;
        }
        TokenType t = currentToken().getType();
        if (t == TokenType.DHORI || t == TokenType.SONGKHA || t ==
            TokenType.BAKKA ||
            t == TokenType.JODI || t == TokenType.DEKHAO || t == TokenType
                .JOTOKKHON || t == TokenType.FOR) {
            return;
        }
        advance();
    }
}
```

A.2 Static Division by Zero Detection in SemanticAnalyzer.java

```
if (op.equals("/") || op.equals("%")) {
    Object rightVal = evaluateConstant(bin.getRight(), scope);
    if (rightVal instanceof Number && ((Number) rightVal).doubleValue() ==
        0.0) {
        String err = "[SEMANTIC ERROR]\nDivision by zero is not allowed.";
        errors.add(err);
        throw new RuntimeException(err);
    }
}
```

```
}
```

APPENDIX B

Representative Borno Programs

B.1 Full Demo Program (valid_full_demo.brn)

```
ধরি সংখ্যা বয়স = ২০;  
ধরি বাক্য নাম = "মাহদি";  
  
যদি (বয়স >= ১৮) {  
    ধরি সংখ্যা অবস্থা = ১;  
    দেখাও("প্রাপ্তবয়স্ক");  
} নাহলে {  
    ধরি সংখ্যা অবস্থা = ০;  
    দেখাও("অপ্রাপ্তবয়স্ক");  
}
```

B.2 While Loop with Nested Condition (TestRunner.java)

```
ধরি সংখ্যা ক = ০;  
  
যতক্ষণ (ক < ৫) {  
    যদি (ক == ২) {  
        দেখাও(ক);  
    }  
    ক = ক + ১;  
}
```